

Computer-Use Automation System

How I approached it, what I decided and why, what I covered — and the walkthrough · interface.ai take-home

An LLM works out how to do a task in a legacy banking UI **once**. What it learned is recorded as a typed, versioned **capability artifact**. That artifact then replays **deterministically, with no model in the loop**, in about two seconds. Anything irreversible — or anything the system has not seen before — stops and hands a person the same live browser session.

STEP 1

The problem in hand

The brief asks for an agent that can operate a legacy enterprise UI. The difficulty is not clicking buttons — Playwright does that. It is that the three things you want are in tension:

You want	But
Flexibility — handle a UI nobody documented, with no API and no test ids	...which argues for an LLM driving the browser live.
Determinism — the same input gives the same result, every time, auditably	...which argues against an LLM anywhere near the hot path.
Safety — some actions move money and cannot be undone	...which argues for a human, but a human on every run is not automation.

Running the model on every execution loses determinism, costs money per run, and leaves nothing to audit. Hand-writing scripts loses the flexibility that made the LLM worth having. So the central decision is *where the model sits*.

SAY

*"My answer is that the model is a **compiler, not a runtime**. It runs once, at authoring time, under policy and with a human available. What it produces is an artifact a person can review and approve — and that artifact is what runs in production, with no model in it at all."*

Why I built the target application

The interesting behaviour is what happens when things go *wrong*: a session expires mid-flow, an undeclared dialog appears, the server 500s, a page is slow, the operator lacks permission. Only an application you control can produce each of those on demand and repeatably.

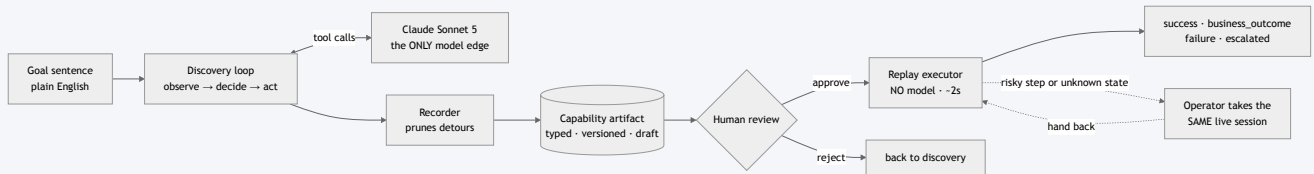
Legacy CU Core is a deliberately dated core-banking UI — framesets, table layouts, no test ids, a native `confirm()` at the point of no return — with synthetic data only and a fault-injection seam the demo drives. That last detail matters more than it looks: the `confirm()` turned out to be the hardest thing in the project,

and I only found out why by running against it.

STEP 2

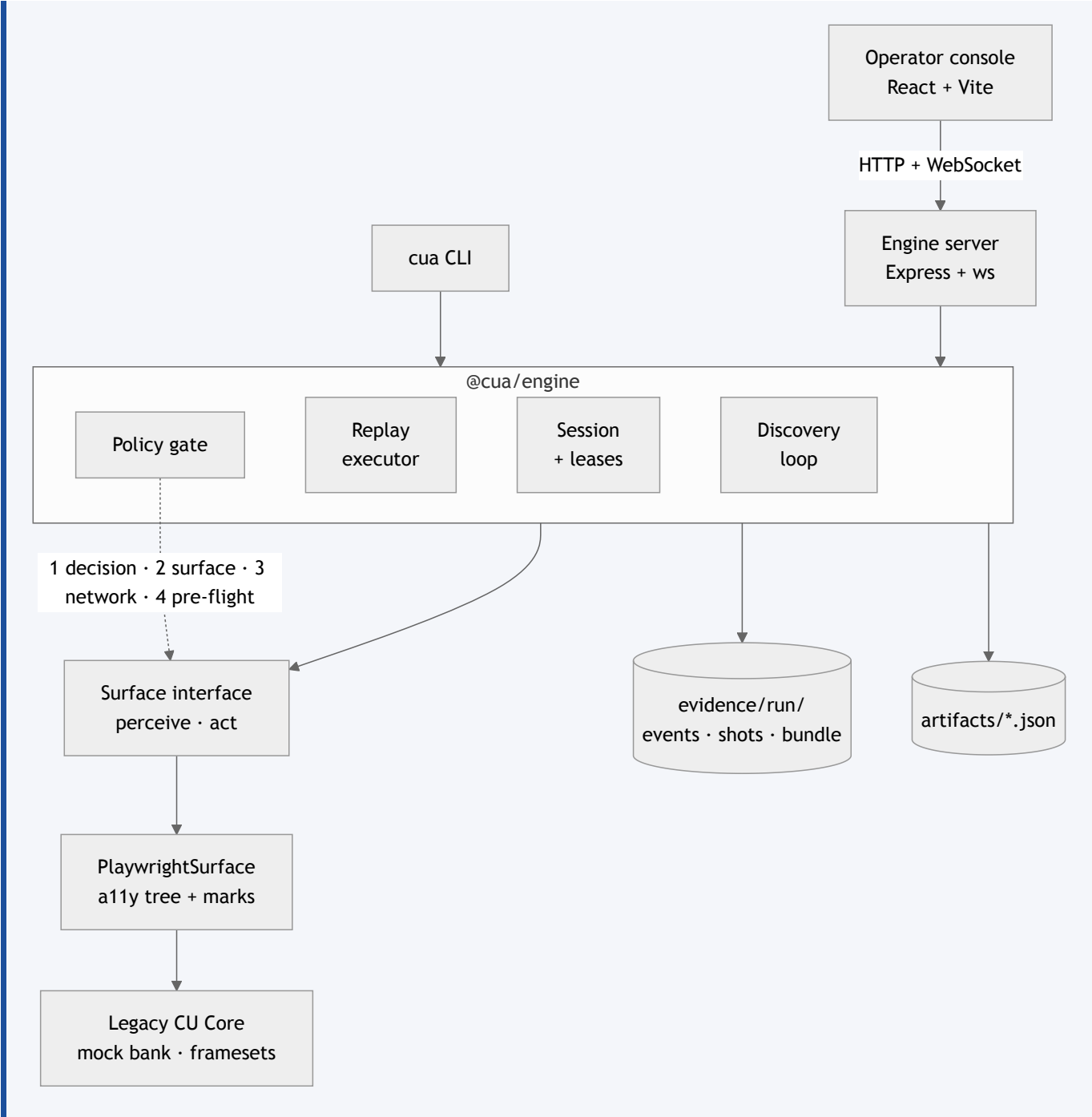
Framework decisions

The life of a capability



The whole argument in one picture: the model sits on a single edge, at authoring time. Everything to the right of the artifact runs without it.

Runtime components



The decisions, and what each one buys

Decision	Alternative I rejected	Why
Learn once, replay forever	Agent drives every run	~\$0.07 and 25s becomes \$0 and 2s, deterministic and auditable. The model's non-determinism is confined to authoring, where a human is already looking.
Artifact is a contract	A recorded action log	Typed inputs/outputs, preconditions, expectations, risk flags, outcome catalog. A reviewer can approve or reject without reading a transcript — which is what makes the approval gate meaningful.
Surface interface	Playwright calls throughout	One seam describing "perceive and act". A desktop surface on an OS accessibility API would implement the same interface; nothing in the artifact references Playwright.
A11y tree + numbered screenshot	Raw HTML in the prompt	Legacy HTML is enormous and mostly layout. The a11y tree is what an operator perceives, flattened across frames; the numbered marks let the model refer to elements by index.
Multi-candidate locators	One selector, or XPath	Role + name, then CSS, then visual position, tried in order. XPath encodes exactly the structure a redesign changes. Matching on a fallback emits a <i>drift</i> event: early warning, not failure.
Screen signatures as preconditions	URL equality	In a frameset the top URL barely changes. A step asserts a URL pattern plus <i>N of M</i> landmarks, so it refuses to run on the wrong screen rather than clicking blindly.
Leases by wrapping	Thread a token through <code>act()</code>	Keeps control-plane concerns out of the perception contract — and revoking a lease disarms every reference at once, <i>including one captured before an await</i> .
Policy as data	Constants in code	One validated <code>policy.yaml</code> answers "what can this touch?" and "what changed?" by reading or diffing one file. Unknown keys are rejected: a misspelled control that silently enforces nothing is worse than a startup error.
Secrets as references	Values in the artifact	Substituted at the browser <i>after</i> the model decides, so a credential never reaches the model, the transcript or the artifact.

Stack

pnpm workspace monorepo · TypeScript strict (engine, CLI, schema, mock app) · Zod schemas with inferred types for every contract · Playwright/Chromium with CDP for per-frame accessibility trees · Express 5 + `ws` for the control plane · React (JavaScript) + Vite for the console, hand-written router, no component library · Vitest, with everything integration-tested against the real mock app and a real browser.

STEP 3

Cases considered

The design decision I am most confident about: **every run ends in exactly one of four shapes, each with its own exit code**. That is what lets a caller automate against the thing instead of parsing logs.

Outcome	Exit	Meaning · who acts
success	0	Task completed, typed outputs returned.
business_outcome	0	The <i>application</i> legitimately said no — no such member, validation rejected, not authorised. The automation worked . A caller branches on the code; nobody is paged at 3am.
failure	2	The automation could not proceed. Always carries a bundle: narrative, viewport and full-page screenshots, accessibility snapshot, page text.
escalated	3	A human is needed — a risky step, an unknown state, or a failure the caller asked to escalate rather than fail.

SAY

"Separating business outcome from failure is what I would defend hardest. 'No such member' is an answer. If that pages someone, the automation is worse than useless, because people start ignoring the alerts."

Recoverable — handled without telling anyone

Condition	What the engine does
SESSION_EXPIRED	Re-runs the login prelude, then restarts the flow from where it lost its place.
SERVER_ERROR	Reloads and retries, bounded.
MAINTENANCE_NOTICE	Dismisses a dialog that <i>is</i> in the outcome catalog.
SLOW_LOAD	Tolerates it, extends the wait, flags it as a soft signal.
RECOVERY_LOOP	The guard. The same recoverable condition twice at the same step stops the run rather than retrying forever.

Hard failures — deliberately debuggable

WRONG_SCREEN · LOCATOR_NOT_FOUND (lists every strategy tried and what was actually visible) · CHECKPOINT_FAILED (the action ran, the expected state did not appear) · UNKNOWN_DIALOG · STEP_TIMEOUT / RUN_TIMEOUT · UNSAFE_RESTART (refuses to restart a flow that already committed something).

Refused before a browser opens

INVALID_ARTIFACT · INVALID_INPUT (a parameter failing its declared pattern) · MISSING_SECRET · ARTIFACT_NOT_APPROVED · POLICY_VIOLATION · RUN_IN_PROGRESS . A bad call costs milliseconds, and stepsRun: 0 proves nothing touched the application.

Risk has two grades, not one

The sub-account flow posts twice: `Continue` validates and re-renders; `Open Account` moves money. Treating both as risky asks a human to approve a step that commits nothing — and an operator asked to approve noise clicks through the prompt that matters at the same speed. So a plain form submit is graded `sideEffect: possible` and runs; only an irreversible signal escalates.

The human-in-the-loop cases

- **Risky step** — approve, and the engine proceeds under its own controller with the approver recorded.
- **Unknown state** — take control of the live browser, fix it by hand, hand back, and the run resumes *at the step it stopped at*.
- **Manual pause** — stop a healthy run between steps, with nothing wrong, to look around.
- **Timeout** — an intervention nobody claims ends the run as `escalated` rather than hanging forever.
- **Conflict** — a second operator claiming the same intervention gets a conflict, not a second lease.

STEP 4

Extra things I took care of, beyond the requirement

Every one of these came from running the system against the real application, not from planning. That is the point worth making.

The goal that was impossible, with the test suite green

The application takes its final approval with a native `confirm()`. The click **fails** with the dialog still open; accepting the dialog is what opens the account. The model had no tool that could answer a dialog — so goal G2 could not be completed at all, and nothing said so. Adding the tool exposed **five further defects**, each producing a plausible wrong answer rather than a crash:

1. The recorder dropped the commit step entirely (it skipped actions that reported failure), producing an artifact that replays straight past the confirmation — and validates cleanly.
2. `pointOfNoReturn` was only ever set on a risky *click*, so a replay that opened a real account would have reported `sideEffects: "possible"`.
3. A modal is not a screen: the recorded precondition was a landmark signature of the page behind the dialog, which cannot be queried while it is blocked — `WRONG_SCREEN`, one action short of the commit.
4. The dialog matched `UNKNOWN_DIALOG` on the very step whose job is to answer it.
5. Replay treated the dialog-raising click as a surface error and stopped before the step that answers it.

SAY

"This is the finding I am proudest of, and it is not a feature. A whole goal was unreachable and every test passed. It was only found by driving the real application to its point of no return and printing what came back."

Accepting a dialog is its own decision

I considered carrying the approval forward so the operator is asked once. I decided against it: the `confirm()` is the application's own last-chance warning, and an operator who approved a click has not yet seen the sentence "This action cannot be undone". Two interventions, each with its own screenshot, is also better evidence than one plus a silent auto-accept.

A monitoring signal that fired on a page that had not changed

Visual drift was intersection-over-union of a control's box against its recording. It reported drift on every control of an unchanged page — legacy buttons are $\sim 37 \times 14$ px and IoU collapses at that scale. Replaced with centre displacement over a scale floor. **Nothing failed: the finding came from reading the evidence of a passing run and asking why it had anything to report.** A signal that is always on is worse than no signal, and it ships, because the tests stay green.

Three separate redaction leaks

The last wrote `interventions.json` unredacted, putting a member ID into a captured URL. The fix was a test that walks **every file a handover writes** rather than the files someone remembered. The remaining limit is stated rather than hidden — see Step 6.

Derive state, do not store it

Run status was a stored field and stuck on "running" forever, because teardown fires a change callback while the state is still running. Made it a getter. That same shape — a stored value clobbered by a callback during teardown — caused four separate bugs, so it became a rule.

Tests that modified the fixtures they were reading

Twice. Once when an artifact was approved on disk while the suite read it as a draft fixture; once when the console learned to record artifacts and the suite began overwriting a committed deliverable. Both now use isolated directories, and the second is asserted.

Secrets: asking git, not `.gitignore`

`cua doctor` checks whether git is *tracking* a secret and whether one was ever committed. Those two answers disagree exactly when a file was staged before the ignore rule was written — the case that actually leaks a key — and deleting it later does not help, because history keeps it. Both states are tested against throwaway repositories.

Defence in depth, because each layer is bypassable alone

Layer	Why the one above it is not enough
1 · Decision	—
2 · Surface boundary	Replay has no model , so layer 1 never runs on the path that runs most often.
3 · Network	A redirect or injected asset moves the session without any engine code asking.
4 · Pre-flight	An artifact travels between environments; its policy must be a subset of the one in force where it lands.

A human who takes control is **not** blocked by layer 2 — they are recorded as `policy_override`. Refusing them would disable escalation exactly when it is needed.

Smaller ones worth a sentence each

- **Prelude circularity** — the sign-in screen matches `SESSION_EXPIRED` and is also where the login prelude legitimately starts; that outcome is inert while checking a precondition inside its own prelude.
- **The run clock stops for humans** — time parked in an intervention does not count against the run timeout, or a long handover would fail a run that was never stuck.
- **Extraction anchors are checked for leakage** — an anchor containing the member ID would differ per member, so the recorder rejects it.
- **Dialogs are raced, not awaited** — a modal used to freeze Playwright mid-action; the surface races the action against dialog appearance with a timeout backstop.
- **Detours are pruned** — a wrong turn during discovery is removed from the recorded flow, so the artifact is the path that worked, not the path that was taken.

STEP 5

The demo

Before recording

```
lsof -ti:4100,4200,4300 | xargs kill -9 2>/dev/null # stale process = engine dies silently
pnpm cua doctor # expect 8 ok, 1 warning, 0 failures
pnpm demo # target app + engine + console; Ctrl-C stops all three

rm -f artifacts/member-branch-and-tenure@1.json # the live-learn goal must not exist
# and member-savings-balance@2 must be a DRAFT, or Act 3's refusal beat does not fire
```

Restarting the target app resets account numbering, so every rehearsal starts at `CU-700001`. Record the terminal window only — `.env` holds a live API key.

Act 1 · The system being automated (60s)

DO Open `localhost:4100`, sign in `demo/demo`, search `10042`.

SAY

"A core banking UI that still runs in production and nobody will rewrite. Framesets, table layouts, no API, no test ids. The balance I want is in an unlabelled table cell."

Act 2 · Learning a task (90s)

DO `pnpm cua discover --goal "Look up member {memberId} and read the current balance of their Savings account." --url http://localhost:4100/ --capability-id member-savings-balance --param memberId=10042`
then `pnpm cua artifact summary artifacts/member-savings-balance@1.json`

SAY

"The model never sees raw HTML — an accessibility tree flattened across frames, plus a screenshot with elements numbered. And the output is not a transcript: it is a typed contract, with locator fallbacks and the outcomes it knows how to handle."

Act 3 · Replay, and the approval gate (90s)

DO `pnpm cua replay artifacts/member-savings-balance@2.json --param memberId=10042 → FAILURE ARTIFACT_NOT_APPROVED`, exit 2, in **3ms without opening a browser**. Approve on the console's Capabilities page, then replay for `10042`, then `10077`.

SAY

"A model worked this out once. That does not make it allowed to run unattended. And the second member proves the parameter is real, not baked into the recording."

Act 4 · The three kinds of wrong (2.5 min)

--param memberId=99999	→ BUSINESS_OUTCOME MEMBER_NOT_FOUND	exit
--param memberId=10042 --fault session_expired	→ SUCCESS, recovered	exit
--param memberId=10042 --fault session_expired:sticky	→ FAILURE RECOVERY_LOOP	exit

DO Open evidence/replay-g1-recovery-loop/failure-*.md — expected state, observed state, suggested next action.

Act 5 · A human takes the same live session (3 min)

DO Console → Start: member-savings-balance@2 , member 10042 , Pace “**Watchable**”, Break something → “**Undeclared error on the next click**”.

Pace matters: at full speed the run ends before the live viewport draws a frame.

The engine stops itself: UNKNOWN_DIALOG , state **paused**, "nobody is driving", a **Waiting** badge.

SAY

"It did not guess. That dialog is not in this capability's outcome catalog. An automation that dismisses unknown dialogs will eventually dismiss something expensive."

DO Take control → "you are driving". Dismiss the alert yourself. **Hand back**. Then **Waiting**: resolved · same · by operator-1 .

SAY

"Seven steps — the same as an uninterrupted run. It resumed the step it stopped at, in the session it paused in. The cookies and frames are the ones my own clicks left behind."

Act 6 · Teaching it something new, live (2 min)

DO Console → **Start** → *Discover with a model* → Goal: “**Something else — describe it yourself**”. Type the goal, name it member-branch-and-tenure , pick **Claude**, set memberId=10042 , press Discover.

The {memberId} you type becomes a parameter on its own, and the *Artifact taking shape* panel writes the recipe as the model works. Rehearsed: **25s, 8 calls, ~\$0.074** → 4 steps, login prelude, **two** outputs. Replays in 2.2s; memberId=10077 returns Riverside / 2018-09-30.

This goal was run once to verify, then deleted — the artifact and its evidence are not in the repository and were never committed. So the run on camera is genuinely the first time the system has seen it, and you already know it works, how long it takes, and what it costs.

SAY

"I have never run that goal. It found both values in a different table from the first capability and worked out a re-readable anchor for each — 'the cell to the right of the label Branch'. Twenty-five seconds to learn, two seconds to run, and it generalises to a member it was never shown."

Act 7 · The flow that commits (3 min)

DO Show `evidence/discovery-g2-open-subaccount/run.json` (15 steps, 15 calls, \$0.14) and `interventions.json` — **three** interventions. Then on Capabilities, steps 7 and 8 of `member-open-subaccount@1`, both *risky · point of no return*.

SAY

"Two of those were the policy. The first one the model raised itself, before any rule fired — it reached the review screen and asked for a human on its own."

```
pnpm cua replay artifacts/member-open-subaccount@1.json --param memberId=10042
#  SUCCESS  {"confirmationNumber":"CU-700001"}  side effects: committed  exit 0
pnpm cua replay artifacts/member-open-subaccount@1.json --param memberId=10042 --risky block
#  FAILURE  POLICY_VIOLATION                    side effects: none      exit 2
```

SAY

"Same capability, same inputs, one line of policy, and the difference is whether an account gets opened. The guardrail belongs to the deployment, not to the recorded flow."

Act 8 · Safety and evidence (90s)

DO `pnpm cua doctor`, then `policy.yaml`, then `evidence/`.

SAY

"One validated file, four enforcement layers, and a secret check that asks git rather than `.gitignore`. Then the honest part — screenshots are not masked."

Disclaimers, limits and cuts

Stating these plainly is deliberate. Every one is written down in the repository as well.

Known limits of what is built

Limit	Detail, and what would fix it
Screenshots are not masked	They render whatever was on screen, member IDs included. The structured evidence <i>is</i> redacted and tested; a grep cannot see inside a PNG. The schema types <code>maskEvidenceScreenshots</code> as the literal <code>false</code> so no deployment can claim masking that does not exist. Every element already has a recorded bbox, so blurring is contained follow-up work.
Console has no authentication	It binds to loopback and assumes one trusted operator. Multi-operator control needs identity; the intervention record already carries <code>approvedBy</code> .
Evidence is unencrypted, no retention	Written to the local filesystem. A real deployment wants an encrypted bucket and an expiry policy.
One application, one tenant	Multi-tenant overlays are designed and specified, not built. The artifact carries the origins it was recorded against, and a different origin is treated as a different tenant.
Cost figures are estimates	Computed from token counts at published prices, recorded per run in <code>usage.json</code> .

Deliberately deferred, with reasons

- **New-tab and detached-element handling** — the page-switch guard and per-step re-resolution are in; a dedicated multi-tab case has no instance in either goal flow.
- **Checkbox state-assert and fill fallbacks** — same reason: no instance in the flows built.
- **Capability catalog API, confidence and stability scoring** — designed and listed as cuts. The schema already carries what they need: status, version, provenance, `approvedBy`.
- **LLM-assisted replay recovery** — deliberately not built. Putting a model back into the replay path would undo the property the whole design exists to provide.

What the demo is, and is not

- The target is a **mock I wrote**. It is representative of legacy patterns, not a real core banking system, and holds synthetic data only — which is why these screenshots are safe to publish.
- The "operator" approving interventions is a human acting through the console's own HTTP API; every approval is recorded with who gave it.
- Replaying the sub-account capability **really opens an account** in the mock. The confirmation number changes every run; that is the proof, not a glitch.
- The live-learned capability in Act 6 is deleted between rehearsals on purpose, so the recorded run is a genuine first encounter.

Questions to expect

Question	Answer
"What if the UI changes?"	Locator fallbacks absorb small changes and emit drift as early warning. A structural change fails with a bundle, and re-discovery produces a new artifact <i>version</i> — a diff, not a mystery.
"Why not run the agent every time?"	Cost, latency, non-determinism, and no compliance story. \$0.07 and 25s versus \$0 and 2s.
"How do you stop it doing something dangerous?"	Four layers from one policy file, an approval gate before unattended replay, and two risk grades so the escalation that matters is not buried in noise. Demonstrated by changing one flag.
"How does this scale past three capabilities?"	Honest answer: catalog API and stability scoring are designed, not built — said plainly rather than hand-waved.
"What went wrong building it?"	Lead with G2 being impossible while the suite was green. It is the strongest thing you can say, because it shows the system was tested against reality.

Numbers worth knowing

Thing	Value
Discovery — G1, read-only	8 calls, ~\$0.072
Discovery — G2, commits, 3 escalations	15 steps, 15 calls, \$0.145
Discovery — live-learned capability	25s, 8 calls, ~\$0.074
Replay — G1 / new / G2	7 steps ~2.0s · 8 steps ~2.2s · 12 steps ~4.0s
Tests	156 engine & CLI + 10 console, against the real mock app and a real browser
Evidence directories	16, indexed from the run files themselves by <code>cua_evidence_index</code>

Print to PDF from Chrome: `Cmd + P` → Destination *Save as PDF* → Background graphics **on**. Be online the first time so the diagram renders.